Studix — Enterprise Master Technical Specification

AI-Powered Academic Resource Sharing Platform & Admin Operating System

Target Agents: Google Antigravity, Cursor, Claude Code | Framework: Spec-Driven Development (SDD)

1. SYSTEM ARCHITECTURE & DUAL-PANEL DESIGN

Studix is a decoupled, modern multi-tenant academic resource sharing engine powered by Supabase (PostgreSQL, Auth, RLS, Storage) and OpenRouter (Gemini API / Multi-LLM provider). The app strictly segregates the **User Panel** and **Admin Panel** via role-based access controls (RBAC).

- **Application Navigation Flow:** Splash Screen → Welcome Screen → Authentication (Login/Signup/Forgot Pass) → College Context Selection (College → Department → Year → Semester) → User Dashboard.
- **User Panel Controls:** Question Papers (Mid-1, Mid-2, Sem), Study Notes (Unit/Faculty/Student), Materials (PDFs, Lab Manuals, PPTs), Interactive Multi-turn AI Assistant, Global Academic Search, Upload Stream with AI Content Moderation, Bookmarks, and Multilingual Voice Search.
- **Admin Panel Controls:** System Analytics Dashboard, College & Department Hierarchy Management, User & Access Governance, Upload Moderation Queue (Approve/Reject files), AI Content Filter Logs, and System Audit Reports.

2. PRODUCTION TECH STACK & MULTILINGUAL STRATEGY

- **Frontend:** React 19, Vite, Redux Toolkit, React Router v6, Tailwind CSS, Framer Motion, Axios, i18next (English, Telugu, Tamil UI & AI response translation), Web Speech API (Speech-to-Text / Text-to-Speech).
- **Backend:** Node.js, Express.js (Modular Monolith architecture with clean Services, Repositories, and Validations).
- **Database & Security:** Supabase PostgreSQL, Supabase Auth (JWT), Row Level Security (RLS), Supabase Storage Buckets (`academic-resources`).
- **AI Engine:** OpenRouter API client hosting Gemini 2.0 Flash / Pro models for automated document validation, multi-turn exam answer synthesis, and repository-aware RAG search.

3. POSTGRESQL RELATIONAL DATABASE SCHEMA & ROW LEVEL SECURITY (RLS)

Execute this complete SQL DDL in your Supabase SQL Editor to initialize all tables, foreign keys, composite indexes, and RLS policies.

```sql
-- 1. ENUMS & EXTENSIONS
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

CREATE TYPE user_role AS ENUM ('STUDENT', 'FACULTY', 'ADMIN', 'SUPER_ADMIN');
CREATE TYPE resource_type AS ENUM ('PREVIOUS_PAPER', 'MID_1', 'MID_2', 'SEMESTER_PAPER', 'INTERNAL_PAPER', 'SUBJECT_NOTES', 'UNIT_NOTES',
'FACULTY_NOTES', 'STUDENT_NOTES', 'LAB_MANUAL', 'PPT', 'ASSIGNMENT', 'REFERENCE_MATERIAL');
CREATE TYPE upload_status AS ENUM ('PENDING', 'APPROVED', 'REJECTED');

-- 2. CORE PLATFORM TABLES
CREATE TABLE colleges (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(255) NOT NULL,
    code VARCHAR(50) UNIQUE NOT NULL,
    domain VARCHAR(100) UNIQUE NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE departments (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    college_id UUID REFERENCES colleges(id) ON DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    code VARCHAR(20) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    UNIQUE(college_id, code)
);

CREATE TABLE users (
    id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
    email VARCHAR(255) UNIQUE NOT NULL,
    full_name VARCHAR(150) NOT NULL,
    role user_role DEFAULT 'STUDENT',
    college_id UUID REFERENCES colleges(id),
    department_id UUID REFERENCES departments(id),
    academic_year INT CHECK (academic_year BETWEEN 1 AND 4),
    semester INT CHECK (semester BETWEEN 1 AND 8),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE subjects (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    department_id UUID REFERENCES departments(id) ON DELETE CASCADE,
    year INT NOT NULL,
    semester INT NOT NULL,
    name VARCHAR(255) NOT NULL,
    code VARCHAR(50) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE resources (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    college_id UUID REFERENCES colleges(id) ON DELETE CASCADE,
    department_id UUID REFERENCES departments(id) ON DELETE CASCADE,
    subject_id UUID REFERENCES subjects(id) ON DELETE CASCADE,
    year INT NOT NULL,
    semester INT NOT NULL,
    title VARCHAR(255) NOT NULL,
    resource_type resource_type NOT NULL,
    file_url TEXT NOT NULL,
    file_path TEXT NOT NULL,
    file_hash VARCHAR(64) NOT NULL, -- SHA-256 for duplicate detection
    ocr_extracted_text TEXT,
    uploaded_by UUID REFERENCES users(id) ON DELETE SET NULL,
    status upload_status DEFAULT 'PENDING',
    approved_by UUID REFERENCES users(id),
    rejection_reason TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE ai_chats (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(255) DEFAULT 'New AI Session',
    subject_id UUID REFERENCES subjects(id),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE ai_messages (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    chat_id UUID REFERENCES ai_chats(id) ON DELETE CASCADE,
    sender VARCHAR(10) CHECK (sender IN ('user', 'assistant')),
    message TEXT NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE TABLE bookmarks (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    resource_id UUID REFERENCES resources(id) ON DELETE CASCADE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    UNIQUE(user_id, resource_id)
);

CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(255) NOT NULL,
    message TEXT NOT NULL,
    is_read BOOLEAN DEFAULT FALSE,
```

```
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 3. INDEXES FOR HIGH-SPEED QUERYING
CREATE INDEX idx_resources_search ON resources (college_id, department_id, year, semester, status);
CREATE INDEX idx_resources_hash ON resources (file_hash);
CREATE INDEX idx_ai_messages_chat ON ai_messages (chat_id, created_at);

-- 4. ROW LEVEL SECURITY (RLS) POLICIES
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
ALTER TABLE resources ENABLE ROW LEVEL SECURITY;
ALTER TABLE ai_chats ENABLE ROW LEVEL SECURITY;
ALTER TABLE ai_messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE bookmarks ENABLE ROW LEVEL SECURITY;

-- Resources: Everyone can read APPROVED resources; Uploaders & Admins can read all
CREATE POLICY "Public Approved Resources" ON resources FOR SELECT USING (status = 'APPROVED');
CREATE POLICY "Student Upload View" ON resources FOR SELECT USING (auth.uid() = uploaded_by);
CREATE POLICY "Admin Full Access" ON resources FOR ALL USING (
  EXISTS (SELECT 1 FROM users WHERE users.id = auth.uid() AND users.role IN ('ADMIN', 'SUPER_ADMIN'))
);

-- AI Chats: Strictly private to session owner
CREATE POLICY "Owner AI Chats Access" ON ai_chats FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "Owner AI Messages Access" ON ai_messages FOR ALL USING (
  EXISTS (SELECT 1 FROM ai_chats WHERE ai_chats.id = ai_messages.chat_id AND ai_chats.user_id = auth.uid())
);
```

4. RESTFUL API SPECIFICATIONS

Method	Endpoint	Description	Auth / Role	Body / Request Details
POST	/api/v1/auth/signup	Register user & create profile	Public	{ email, password, fullName, collegeId, departmentId, year, sem }
POST	/api/v1/resources/upload	Upload resource with AI validation	JWT (Student+)	Multipart form: file, subjectId, resourceType, title
GET	/api/v1/resources	Query shared repository	JWT (Student+)	Params: collegeId, departmentId, year, semester, search
POST	/api/v1/ai/validate	AI pre-upload filter worker	Server Internal	Validates file stream; rejects non-academic photos/documents
POST	/api/v1/ai/paper-analysis	Multi-turn exam solver prompt setup	JWT (Student+)	{ resourceId, questionSelection, marks, format, explanationStyle }
POST	/api/v1/ai/repository-search	Repository-aware RAG query	JWT (Student+)	{ query, collegeId, departmentId, subjectId }
GET	/api/v1/admin/moderation	Fetch pending resource uploads	JWT (Admin)	Filters: PENDING, collegeId, departmentId
PATCH	/api/v1/admin/resources/:id	Approve or reject resource	JWT (Admin)	{ status: "APPROVED" | "REJECTED", rejectionReason }

5. MONOREPO DIRECTORY BLUEPRINT

```
studix/
├── client/                        # React + Vite Frontend App
│   ├── public/
│   │   └── locales/               # i18n Translation Json Files (en, te, ta)
│   ├── src/
│   │   ├── assets/
│   │   ├── components/
│   │   │   ├── common/            # Navbar, Sidebar, Modal, Loader, VoiceButton
│   │   │   ├── user/              # ResourceCard, PDFViewer, AIModal
│   │   │   └── admin/             # AnalyticsCard, ModerationTable, UserTable
│   │   ├── layouts/               # UserLayout, AdminLayout, AuthLayout
│   │   ├── pages/
│   │   │   ├── auth/              # Login, Signup, ForgotPassword
│   │   │   ├── onboarding/        # SelectCollege, SelectDept
│   │   │   ├── user/              # Dashboard, Papers, Notes, AIAssistant, Bookmarks
│   │   │   └── admin/             # Dashboard, ModerationQueue, UserManagement
│   │   ├── redux/                 # Store, authSlice, academicSlice, aiSlice
│   │   ├── services/              # api.js, supabaseClient.js, speechToText.js
│   │   ├── i18n/                  # i18next configuration setup
│   │   ├── App.jsx
│   │   └── main.jsx
│   └── package.json
│
├── server/                        # Express.js Backend Engine
│   ├── config/                    # Supabase, OpenRouter SDK Setup
│   ├── controllers/               # authController, resourceController, aiController, adminController
│   ├── middleware/                # authMiddleware, adminMiddleware, uploadMiddleware
│   ├── services/                  # openRouterService, duplicateDetectionService, ocrService
│   ├── validations/               # resourceValidation, authValidation
│   ├── app.js
│   └── server.js
│
├── database/
│   └── schema.sql                 # Complete PostgreSQL + RLS Script
├── .env.example
└── package.json
```

6. PHASED IMPLEMENTATION ROADMAP FOR GOOGLE ANTIGRAVITY

PROMPT 1 **Phase 1: Foundation, Supabase Core & Multi-Tenant Onboarding**

```
Initialize full monorepo structure for studix (client React + Vite, server Express). Execute schema.sql in Supabase to setup PostgreSQL tables (users, colleges, departments, subjects, resources) and RLS. Build authentication APIs and UI (Login, Signup, Forgot Password). Implement onboarding flow (Select College -> Department -> Year -> Semester) persisting choices to Redux toolkit and session storage. Ensure complete TypeScript/JS types and zero placeholder code.
```

PROMPT 2 **Phase 2: Academic Repository, Upload Pipeline & Duplicate Detection**

```
Execute Phase 2: Create POST /api/v1/resources/upload endpoint. Implement Multer file ingestion and calculate SHA-256 hash to reject duplicate files before storage. Connect OpenRouter API (Gemini Vision/Text model) to automatically inspect uploaded documents: approve academic papers/notes, reject non-academic photos/selfies with "This file is not a valid academic resource." Build User Panel repository views (Previous Papers, Mid-1/Mid-2, Notes, Materials) with global multi-filter search and bookmarking.
```

PROMPT 3 **Phase 3: Multi-turn Exam AI Assistant & Repository-Aware RAG**

```
Execute Phase 3: Implement OpenRouter Gemini RAG engine. Build repository-aware search: AI queries Supabase to check if specific papers/notes exist for a college/subject. Build interactive multi-turn paper analysis flow: when paper is uploaded, AI analyzes content and asks follow-up preference questions (Marks: 2/5/10/16, format: bullet/diagram/university style) before synthesizing step-by-step solutions. Implement complete RLS-enforced private AI chat messaging.
```

PROMPT 4 **Phase 4: Admin Operating Panel, Multilingual Voice & Security Hardening**

```
Execute Phase 4: Build standalone Admin Panel with role checks (ADMIN/SUPER_ADMIN). Create Admin Moderation Queue dashboard to approve/reject resources and view AI rejection logs. Integrate i18next supporting English, Telugu, and Tamil with Web Speech API (Speech-to-Text and TTS). Apply Helmet, CORS, Express rate limiting, and prepare production scripts for Vercel/Render deployment.
```